

JavaScript Variables

Variables are the foundation of storing and working with data in JavaScript.

1. What Is a Variable?

A **variable** is a named container used to store a value in memory.

Think of a variable like a **labeled box**:

```
name  —————> "Rokon"  
age   —————> 23
```

The **variable name** gives us a way to access the stored value later.

Example

```
let name = "Rokon";  
const age = 23;  
  
console.log(name);  
console.log(age);
```

Output:

```
Rokon  
23
```

Here:

- `name` is the variable.
- `"Rokon"` is the value stored in `name`.
- `age` is the variable.
- `23` is the value stored in `age`.

The basic structure is:

```
variable = value;
```

| 2. Why Do We Need Variables?

Without variables, we would have to write the same data repeatedly.

| Without a Variable

```
console.log("Rokon");  
console.log("Rokon");  
console.log("Rokon");
```

This works, but it becomes difficult to maintain when the value needs to change.

| With a Variable

```
let name = "Rokon";  
  
console.log(name);  
console.log(name);  
console.log(name);
```

Now the value is stored in one place.

If the value changes:

```
let name = "Rahim";  
  
console.log(name);
```

Output:

```
Rahim
```

This makes the code **cleaner, reusable, and easier to maintain**.

3. Declaring Variables in JavaScript

JavaScript provides three keywords for declaring variables:

```
var  
let  
const
```

Although all three can create variables, they behave differently.

The most important differences are:

- Can the value be changed?
- Can the variable be declared again?
- What type of scope does it have?

4. `var`

`var` is the **older way** of declaring variables in JavaScript.

| Basic Syntax

```
var variableName = value;
```

| Example

```
var name = "Rokon";  
  
console.log(name);
```

Output:

```
Rokon
```

| Reassignment

A `var` variable can be assigned a new value.

```
var age = 23;

age = 24;

console.log(age);
```

Output:

```
24
```

The value changed from 23 to 24 .

| Redeclaration

A `var` variable can also be declared again in the same scope.

```
var name = "Rokon";

var name = "Rahim";

console.log(name);
```

Output:

```
Rahim
```

So, `var` allows both:

```
Reassignment ✓
Redeclaration ✓
```

Modern JavaScript generally avoids `var` because its behavior can lead to unexpected results.

5. let

`let` is the **modern way** to declare a variable when its value may change.

| Basic Syntax

```
let variableName = value;
```

| Example

```
let age = 23;  
  
console.log(age);
```

Output:

```
23
```

| Reassignment

A `let` variable can be assigned a new value.

```
let age = 23;  
  
age = 24;  
  
console.log(age);
```

Output:

```
24
```

So:

```
23 → 24
```

The variable remains the same, but its stored value changes.

| Redeclaration

A `let` variable **cannot be redeclared in the same scope**.

```
let age = 23;  
  
let age = 24;
```

This produces:

```
SyntaxError
```

Therefore:

```
Reassignment ✓  
Redeclaration ✗
```

6. `const`

`const` is used when a variable should **not be reassigned** after it has been initialized.

| Basic Syntax

```
const variableName = value;
```

| Example

```
const PI = 3.1416;  
  
console.log(PI);
```

Output:

```
3.1416
```

| Reassignment

A `const` variable cannot receive a new value.

```
const PI = 3.1416;  
  
PI = 3.14;
```

This produces:

```
TypeError
```

Therefore:

```
Reassignment  X  
Redeclaration X
```

| Important Rule

A `const` variable must be initialized when it is declared.

```
const name = "Rokon";
```

This is valid.

But:

```
const name;
```

is invalid because no value was assigned during declaration.

7. var vs let vs const

This is one of the most important comparisons to remember.

Feature	var	let	const
Can reassign?	yes	yes	no
Can redeclare?	yes	no	no
Scope	Function	Block	Block
Must initialize?	yes	no	yes
Modern usage	Rare	Common	Preferred

8. Reassignment vs Redeclaration

These two terms are often confused.

Reassignment

Reassignment means changing the value of an existing variable.

```
let age = 23;  
  
age = 24;
```

Here, the variable `age` already exists.

We simply changed its value:

```
23 → 24
```

This is called **reassignment**.

| Redeclaration

Redeclaration means declaring the same variable again.

```
let age = 23;  
  
let age = 24;
```

The second `let age` tries to create the same variable again.

That is **redeclaration**, and it is not allowed for `let`.

9. A Simple Comparison

| var

```
var x = 10;  
  
x = 20;  
var x = 30;  
  
console.log(x);
```

Output:

```
30
```

`var` allows both reassignment and redeclaration.

let

```
let x = 10;  
  
x = 20;  
  
console.log(x);
```

Output:

```
20
```

let allows reassignment.

But this is not allowed:

```
let x = 10;  
let x = 20;
```

const

```
const x = 10;  
  
console.log(x);
```

Output:

```
10
```

But this is not allowed:

```
const x = 10;

x = 20;
```

10. Which One Should You Use?

In modern JavaScript, follow this simple rule:

| Use **const** by default

If the variable does not need to be reassigned:

```
const name = "Rokon";
const country = "Bangladesh";
const birthYear = 2003;
```

| Use **let** when the value changes

```
let score = 0;

score = 10;
score = 20;
```

| Avoid **var**

```
var name = "Rokon";
```

Although **var** is still valid JavaScript, modern JavaScript code generally prefers **let** and **const**.

11. The Easiest Way to Remember

Remember this single rule:

```
const → Cannot reassign  
let   → Can reassign  
var   → Can reassign + redeclare
```

Or visually:

Keyword	Reassign	Redeclare
var	✓ Yes	✓ Yes
let	✓ Yes	✗ No
const	✗ No	✗ No

12. Best Practice

A clean modern JavaScript codebase usually follows this pattern:

```
const name = "Rokon";  
const country = "Bangladesh";  
  
let age = 23;  
  
age = 24;
```

The idea is simple:

```
Value will NOT change → const  
Value WILL change    → let  
Old code / legacy     → var
```

Quick Revision

| Variable

A variable is a named container used to store a value.

`var`

- Old variable declaration method
- Can be reassigned
- Can be redeclared

`let`

- Modern variable declaration
- Can be reassigned
- Cannot be redeclared in the same scope

`const`

- Used for values that should not be reassigned
- Cannot be reassigned
- Cannot be redeclared
- Must be initialized during declaration

Golden Rule

Use `const` by default. Use `let` when the value needs to change. Avoid `var` in modern JavaScript.